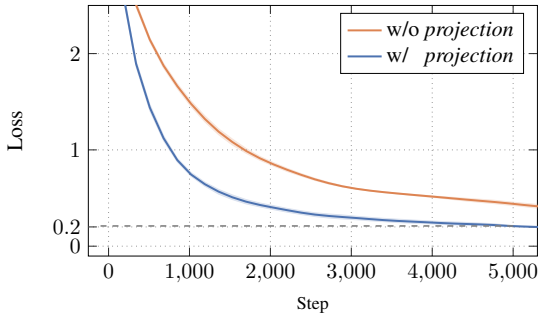
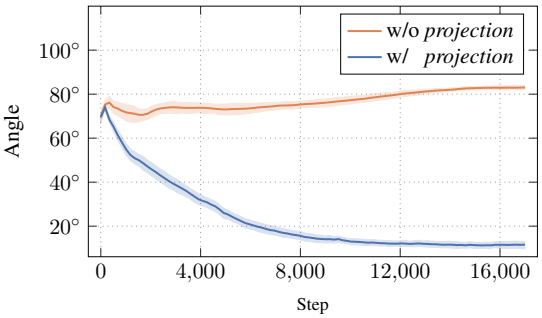


(a) Training loss: *With* projection, the model converges faster compared to the model *without* projection, which required 3x more steps.



(b) The mean angle of the queries to the $\vec{\mathbb{1}}$ vector. *With* projection, since all keys are orthogonal to $\vec{\mathbb{1}}$, the model has learned to align the queries to be parallel to $\vec{\mathbb{1}}$ and thus give equal attention to all keys.

Figure 2: The training loss and mean angle of the queries to $\vec{\mathbb{1}}$ in the "majority" task across 10 runs with and without *projection*.

We experimented with standard LayerNorm compared to a LayerNorm without *projection* (having a numerator of simply $\boldsymbol{x}$ in Equation (1), similarly to the LayerNorm variant of Zhang and Sennrich (2019)). We trained each model 10 times using different random seeds.

Results Figure 2a shows that *with* projection, the model converges faster compared to *without* projection. We hypothesize that since all key vectors are orthogonal to the $\vec{\mathbb{1}}$ vector, the model can exploit this geometric structure, and learning the task is made easier. Figure 2b shows that indeed, the model *with* projection has learned to align the queries to the $\vec{\mathbb{1}}$ vector, decreasing the angle between the queries and $\vec{\mathbb{1}}$. On the contrary, a model *without* projection has to learn to solve this task "from scratch". This model also converged eventually, but it required 3x more training steps. Figure 3 shows a similar trend in the models' test accuracies.

4.2 Unselectable Keys

We examined the fraction of "unselectable" keys in a Transformer model with and without the *scaling* component of LayerNorm using the method presented in Grivas et al. (2022). We trained a 4-layer language model (based on GPT2 architecture (Radford et al., 2019)) with $d = 8$ on Wikipedia [3] for 50K steps, and analyzed the inputs to the attention layer in each layer using sequences from the validation set of SQuAD (Rajpurkar et al., 2016).

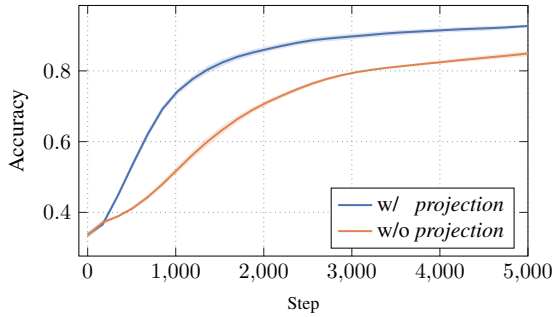


Figure 3: The test accuracy in the "majority" task across 10 runs with and without *projection*. *With* projection the model reaches high test accuracy faster compared to the model *without* projection.

Model	L_1	L_2	L_3	L_4
w/o *scaling*	51.0	32.2	34.7	36.8
w/ *scaling*	**0**	**0**	**0**	**0**

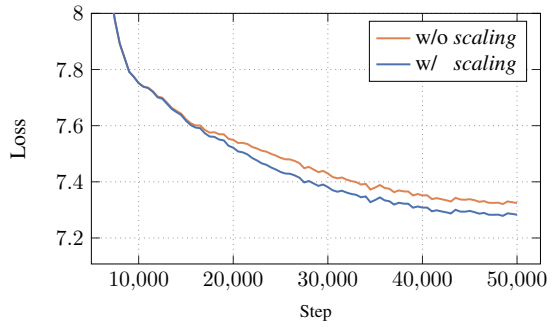
Table 1: The fraction of "unselectable" key vectors right before the attention mechanism of each layer of a language model without *scaling*. LayerNorm solves the "unselectable" keys problem using the *scaling* property. Without *scaling*, there are key vectors that cannot be selected by the attention mechanism.

Results Table 1 shows the following results: *Without scaling*, there are at least 32% "unselectable" keys that cannot receive maximal attention score *in each layer*. In contrast, *scaling* removes this problem and allows the model to potentially focus on any key. In Figures 4a and 4b, we show that this difference in "unselectable" keys is also reflected in higher training and test losses for the model that does not use *scaling*.

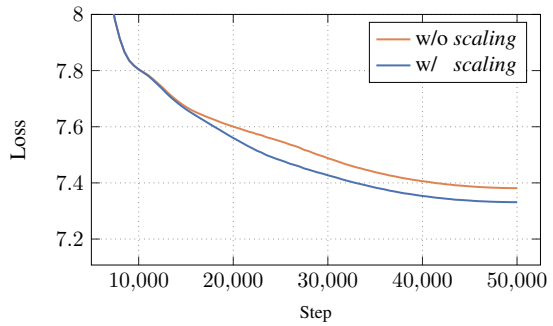
5 Conclusion

In this paper, we show that the commonly used LayerNorm component is crucial not only for the optimization process but also for the expressivity of attention in Transformers. We decompose LayerNorm into two geometric operations: *projecting*

[3] https://huggingface.co/datasets/wikipedia, 20220301.en split.



(a) Training loss: *With* scaling, the model converge faster compared to the model *without* scaling.



(b) Test loss: *With* scaling, the model achieves lower test loss faster compared to the model *without* scaling.

Figure 4: The training and test loss in the language modeling task.

the input vectors onto a subspace that is orthogonal to the $\vec{\mathbb{1}}$ vector, and *scaling* the projected vectors to have the same norm $\sqrt{d}$.

We show that *projection* helps to compute even simple tasks such as "majority" by performing an exact average of the keys and that *scaling* helps to avoid the problem of "unselectable" keys.

These findings are important for the community's understanding of attention and expressivity in Transformers. Further, these results raise a variety of follow-up questions, such as: why should the keys be orthogonal to the $\vec{\mathbb{1}}$ vector, instead of any, learnable, different vector for every layer? And what would happen if we force each layer's keys to be orthogonal to *multiple* normal vectors? To this end, we make our code publicly available at `https://github.com/tech-srl/layer_norm_expressivity_role`.

6 Limitations

In this work, we found that the implications of the geometric properties of LayerNorm affect mainly small models and are less evident for larger models. We hypothesize that with a large hidden dimension, a Transformer model can find other solutions for computing "majority" using gradient descent and is, therefore, less dependent on the *projection* component. Further, we believe that the *scaling* component is less useful for high dimensional models, since with higher dimensions, it is less likely to encounter a set of vectors where some of them lie within the convex hull of the others. Therefore, we encourage the community to use LayerNorm before attention layers, especially for small models that operate on long sequences.

Moreover, the *projection* component is clearly a linear operator that can be expressed by a linear layer before the LayerNorm, as we show in Appendix C. Nevertheless, the importance of the projection holds as we discuss in Section 3, and the benefit of using this operator explicitly in LayerNorm is shown in Section 4.1.

Acknowledgements

We thank Gail Weiss for the helpful discussions.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *ArXiv preprint*, abs/1607.06450.

David Demeter, Gregory Kimmel, and Doug Downey. 2020. Stolen probability: A structural weakness of neural language models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2191–2197, Online. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Wikimedia Foundation. Wikimedia downloads.

Andreas Grivas, Nikolay Bogoychev, and Adam Lopez. 2022. Low-rank softmax can have unargmaxable classes in theory but rarely in practice. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6738–6758, Dublin, Ireland. Association for Computational Linguistics.

Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 448–456. JMLR.org.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. *arXiv e-prints*, page arXiv:1606.05250.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, Seattle, Washington, USA. Association for Computational Linguistics.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. 2020. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533. PMLR.

Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. 2019. Understanding and improving layer normalization. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 4383–4393.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32.

A Proof of Theorem 1

Theorem 1. *Given a set of vectors $\mathcal{S} = \{\boldsymbol{h}_1, ..., \boldsymbol{h}_{n-1}, \boldsymbol{h}_n\}$ such that $\boldsymbol{h}_n$ is interior to the convex hull of $\mathcal{S}$, then for all $\boldsymbol{v} \in \mathbb{R}^d$ (s.t. $\boldsymbol{v} \neq \vec{\mathbf{0}}$):*

$$\max_{i \in [n-1]} \boldsymbol{v}^\top \boldsymbol{h}_i > \boldsymbol{v}^\top \boldsymbol{h}_n$$

We prove Theorem 1 using Theorem 2, presented and proved in Demeter et al. (2020):

Theorem 2 (Demeter et al. (2020))**.** *Let C be the convex hull of the embeddings $\{x_i\}$ of a vocabulary V. If an embedding x_i for word $w_i \in V$ is interior to C, then the maximum probability $P(w_i)$ assigned to w_i using a dot-product softmax is bounded by the probability assigned to at least one word w_i whose embedding is on the convex hull.*

Proof of Theorem 1. According to Theorem 2, since $\boldsymbol{h}_n$ is interior to the convex hull of $\mathcal{S}$, the maximum probability assigned to $\boldsymbol{h}_n$ is bounded by the maximum probability assigned to $\boldsymbol{h}_i$ for some $i \in [1-n]$, that is on the convex hull of $\mathcal{S}$. Since probability in Transformers is computed as a dot-product of the final hidden state of the Transformer $\boldsymbol{u}$ and the embedding vector, we can write:

$$\boldsymbol{u}^\top \boldsymbol{h}_i > \boldsymbol{u}^\top \boldsymbol{h}_n \tag{7}$$

for *any* $\boldsymbol{u} \in \mathbb{R}^d$ (the probability is computed as a *softmax* of the dot-product logits, but since softmax is a monotonic function, a higher probability after the softmax necessarily implies a higher logit score).

Therefore, it also implies that for any $\boldsymbol{v} = \boldsymbol{u} \in \mathbb{R}^d$:

$$\max_{i \in [n-1]} \boldsymbol{v}^\top \boldsymbol{h}_i > \boldsymbol{v}^\top \boldsymbol{h}_n \tag{8}$$

□

B Characteristics of the Normalized Vectors

In this section, we discuss the characteristics of the LayerNorm inputs that are been normalized to the same point. Recall that the *projection* ensures that the normalized output lies on the hyperplane $\mathcal{H}$ defined by the normal vector $\vec{\mathbb{1}} = [1, 1, ..., 1] \in \mathbb{R}^d$.

Let $\boldsymbol{v}$ be a unit vector in $\mathcal{H}$:

$$\boldsymbol{v} \perp \vec{\mathbb{1}} \wedge ||\boldsymbol{v}|| = 1 \tag{9}$$

Therefore

$$\sum_{i=1}^{d} \boldsymbol{v}_i = 0 \tag{10}$$

Let $\mathcal{M}$ be a **2D plane** that is defined using $\boldsymbol{v}$ and $\vec{\mathbb{1}}$. Its parametric representation is:

$$\mathcal{M} : s\boldsymbol{v} + t\vec{\mathbb{1}} \tag{11}$$

Finally, let $\boldsymbol{x}$ be a vector in $\mathcal{M}$. Therefore, there exist $\alpha, \beta \in \mathbb{R}$ such that

$$\boldsymbol{x} = \alpha \boldsymbol{v} + \beta \vec{\mathbb{1}} \tag{12}$$